

1 1.C - Fiabilité des Protocoles

Dans cette section, nous explorerons divers concepts couramment utilisés dans les protocoles, en particulier les protocoles de liaison de données. Nous commencerons par examiner comment définir le début et la fin d'une séquence binaire à transmettre sur une liaison. Une liaison est un support de transmission reliant deux cartes réseau permettant le transfert de signaux. Nous verrons comment reconnaître le début et la fin de la séquence transmise à l'aide d'un signal et comment assurer la transparence des données. Ensuite, nous aborderons le contrôle d'intégrité, c'est-à-dire la détection et la gestion des erreurs de transmission de données à l'aide de sommes de contrôle. Enfin, nous discuterons de la fiabilité, c'est-à-dire des mécanismes et algorithmes permettant de garantir que les données reçues à leur destination soient identiques à celles émises à la source.

1.1 La délimitation des données

1.1.1 Protocole et fiabilité

Un protocole de communication est un ensemble de règles et de conventions définissant comment deux entités peuvent échanger des informations et communiquer entre elles pour fournir un service dans un réseau. Généralement, cela implique l'exécution d'un algorithme de chaque côté et la définition de messages spécifiques permettant de fournir le service attendu.

La fiabilité de la communication se réfère à la qualité de transmission de données de manière à garantir que ce qui arrive à destination est exactement ce qui a été envoyé au départ. Cela signifie vérifier qu'il n'y a pas d'erreurs, c'est-à-dire qu'aucun bit n'a changé de valeur pendant la transmission, qu'il n'y a pas de pertes, c'est-à-dire qu'aucune séquence binaire n'a disparu pendant la transmission, qu'il n'y a pas de duplication de messages (arrivée en double d'un même message), et que les différentes parties transmises arrivent dans le bon ordre. Par exemple, lorsqu'un fichier est divisé en plusieurs morceaux, il est important de garantir que ces morceaux arrivent dans le bon ordre afin que le fichier puisse être reconstitué correctement.

1.1.2 Notion de fanion

Commençons par voir comment définir le début et la fin d'une trame. Une trame est l'élément qui est transmis sur une liaison. Elle correspond à une séquence binaire bien identifiée dont il faut reconnaître le début et la fin lors de la réception du signal par la carte réseau réceptrice. Pour cela, on va utiliser un caractère spécial qui s'appelle le **fanion**. Le fanion est une séquence binaire prédéfinie par le protocole de la liaison, c'est-à-dire qui a une signification particulière au niveau du protocole de la liaison qui va transférer les trames. Cette séquence binaire signifie "*début ou fin de trame*".



Figure 1: Fanion

1.1.3 Notion de transparence : l'échappement

Le fait de définir un caractère spécial comme le fanion pose un problème lorsque cette séquence binaire se retrouve à l'intérieur des données transmises, car dans cette situation, elle ne doit pas être interprétée comme le début ou la fin d'une trame. Pour résoudre ce problème, nous utilisons la technique de l'échappement, qui consiste à définir un nouveau caractère spécial appelé **caractère d'échappement** et à l'insérer devant tout caractère nécessitant être échappé dans les données transmises. Un caractère échappé, c'est-à-dire précédé d'un caractère d'échappement ne sera pas interprété par le protocole en réception. Lorsque les données sont reçues, si le caractère d'échappement est détecté, le caractère suivant n'est pas interprété comme un caractère

spécial et reste simplement une donnée dans les données transmises. Par ailleurs, le caractère d'échappement est supprimé puisqu'il a été ajouté artificiellement par l'émetteur pour faire l'échappement.

Pour effectuer l'échappement, l'algorithme est donc le suivant :

1. Avant de transmettre les données, l'émetteur doit identifier tous les caractères qui doivent être échappés et insérer le caractère d'échappement juste avant chacun d'eux.
2. Lorsque les données sont reçues, le récepteur parcourt les données et, lorsqu'il détecte le caractère d'échappement, il sait que le caractère suivant ne doit pas être interprété comme un caractère spécial et doit simplement être inclus dans les données reçues et transmises à la couche supérieure.

Cette technique permet de garantir que les caractères spéciaux ne sont pas interprétés de manière erronée et que les données reçues sont correctement interprétées.

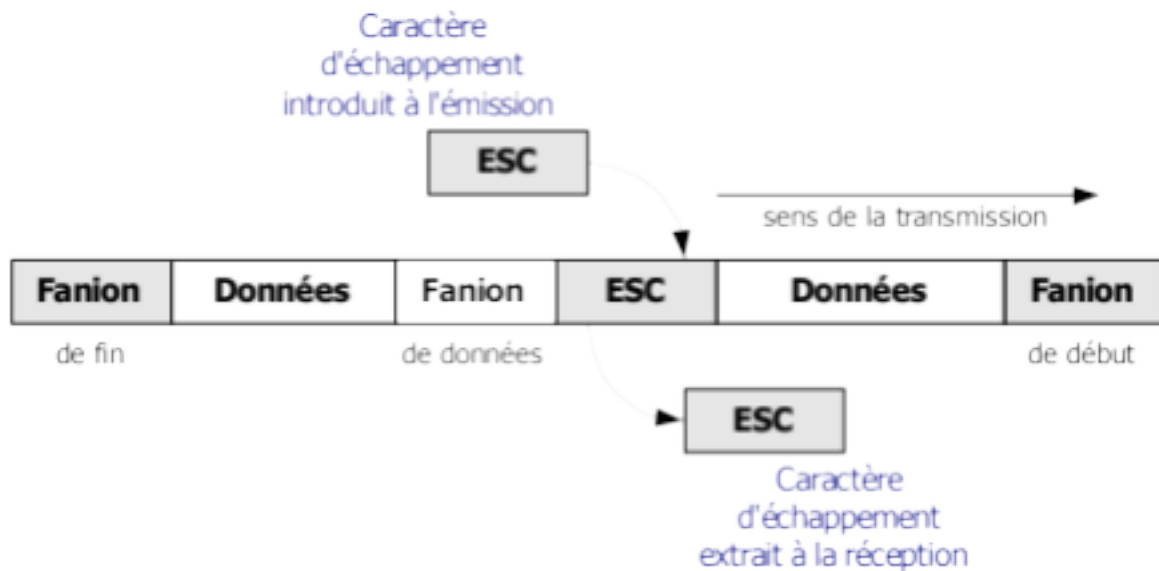


Figure 2: Transparence

Dans cet exemple, vous pouvez voir que le caractère d'échappement a été inséré juste avant la séquence binaire qui, dans ce cas, est identique au fanion mais n'est ni un début ni une fin de trame. Il s'agit d'une donnée comprise dans la trame dont la valeur est identique à celle du fanion. À la réception, ce caractère spécial d'échappement sera retiré.

Il est également possible que le caractère d'échappement se trouve lui-même dans les données et doive donc être échappé à son tour. Dans ce cas, la technique est la même et consiste à ajouter le caractère d'échappement devant l'échappement présent dans les données.

1.1.4 Technique du bit de bourrage

Nous avons vu comment faire l'échappement d'un caractère, c'est-à-dire comment remplacer une séquence binaire sur huit bits qui correspond à un octet par un caractère spécial, comme le caractère échappement ou le caractère fanion. Pour assurer la transparence au niveau d'une trame, on utilise souvent la **technique du bit de bourrage**. Celle-ci consiste à essayer de casser le fanion s'il est présent dans les données, plutôt que d'ajouter un caractère échappement complet. Le caractère fanion correspond à la séquence "01111110". Pour faire l'échappement, il suffit donc d'empêcher qu'il y ait 6 bits à 1 consécutifs dans les données.

Pour ce faire, on ajoute un zéro à l'émission chaque fois qu'on rencontre la séquence binaire zéro suivie de cinq fois un soit "011111", ce qui permet d'éviter d'avoir la séquence correspondant au fanion à l'intérieur de la trame. Sur l'émetteur, on ajoute donc un zéro pour casser le fanion qui se trouve dans les données, et

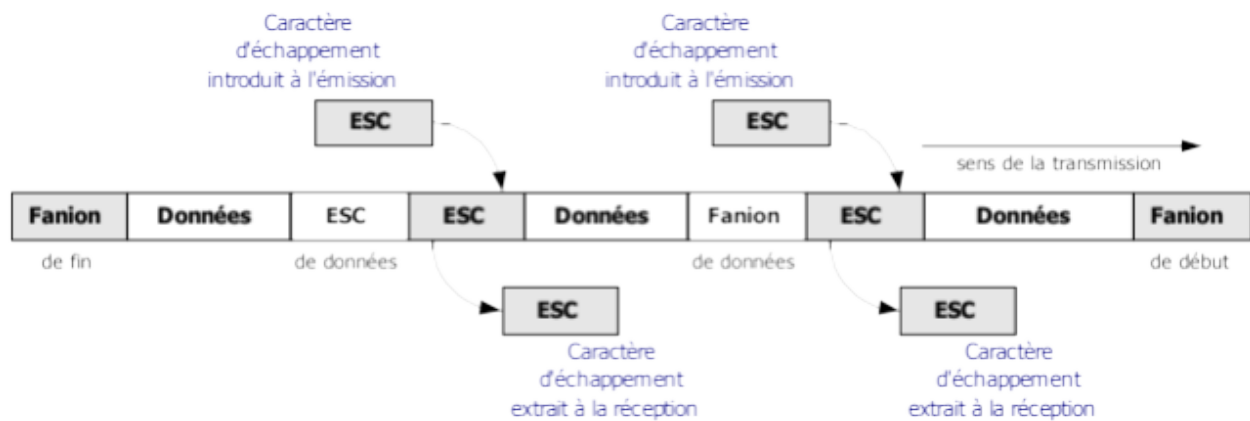


Figure 3: Caractère d'échappement

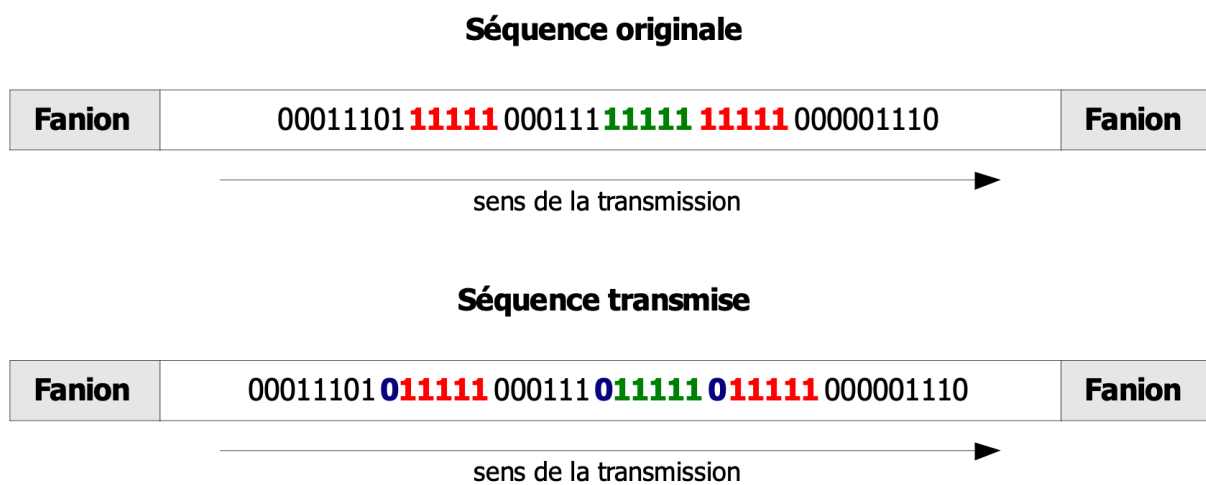


Figure 4: Technique du bit de bourrage

sur le récepteur, on fait l'opération inverse en retirant le zéro ajouté artificiellement par l'émetteur lorsqu'on reçoit la séquence binaire "011111". Cela permet d'assurer la transparence en ajoutant simplement quelques zéros plutôt qu'un caractère d'échappement complet.

1.2 Le contrôle d'intégrité

Passons maintenant à la question du contrôle d'intégrité. Il s'agit de savoir comment détecter les erreurs qui peuvent survenir lors de la transmission de données. Que sont ces erreurs et comment les repérer ?

1.2.1 Notion d'erreur

Une erreur peut avoir plusieurs origines, en particulier le signal lui-même qui peut subir des perturbations d'origine physique liées à des rayonnements, des distorsions, une atténuation du signal ou des interférences extérieures, typiquement liées à des rayonnements électromagnétiques. **Une erreur se produit lorsqu'un bit change de valeur pendant sa transmission**, c'est-à-dire lorsqu'un zéro devient un un ou un un devient zéro. Pour quantifier la probabilité de recevoir des erreurs, on utilise le **taux d'erreur binaire (BER)** (ou Bit Error Rate en anglais) qui est la probabilité pour qu'un bit change de valeur pendant sa transmission. Sa valeur est déterminée suite à de nombreux tests en laboratoire qui consiste à transmettre des séquences binaires selon différentes conditions et à mesurer lors de la réception le nombre de bits erronés sur le nombre total de bits transmis. Par exemple, si une séquence binaire de 16 bits comporte 3 bits qui ont changé de valeur pendant la transmission, le taux d'erreur binaire de cette transmission est de $3/16$, soit 18,75 %. En pratique, le taux d'erreur binaire varie entre 10^{-4} pour les liaisons les moins performantes, 10^{-9} pour les liaisons Ethernet et peut atteindre 10^{-12} sur de la fibre optique, avec des erreurs très rares. Cependant, il est important de noter que les erreurs ont tendance à se produire par rafales, étant donné qu'elles sont liées à une perturbation physique qui interfère sur plusieurs bits consécutifs.

Maintenant, ce qui nous intéresse, c'est de savoir quand on transfère une trame sur une liaison entre deux cartes réseau, quelle est la probabilité qu'elle subisse une erreur ? Le raisonnement est assez simple : une trame est erronée si elle a subi au moins une erreur. A l'inverse, pour qu'une trame soit correcte, il faut que tous les bits soient corrects. La probabilité pour qu'un bit soit correct est par définition $1 - \text{BER}$. Donc la probabilité pour qu'une trame de n bits soit correcte est $(1 - \text{BER})^n$.

Ainsi, la **probabilité qu'une trame soit erronée** est $1 - (1 - \text{BER})^n$, où n est le nombre de bits de la trame. Elle dépend du taux d'erreur binaire (BER) de la liaison sur laquelle elle est transmise ainsi et la longueur de la trame. Plus une trame est grande, plus elle risque de subir une erreur et donc d'être erronée à la réception.

Cette information est importante car elle permet de quantifier la fiabilité d'un protocole en mesurant la perte de débit liée aux erreurs. En effet, la probabilité qu'une trame soit erronée intervient dans le calcul de l'efficacité du protocole.

1.2.2 Détection d'erreur

Pour détecter une erreur, il est nécessaire de disposer d'un élément de comparaison. Une méthode couramment utilisée est la **détection par répétition**, qui consiste à envoyer le message en double depuis l'émetteur ou à demander au récepteur de renvoyer un écho du message. Cependant, cette méthode a plusieurs inconvénients. Premièrement, elle est peu efficace car elle utilise beaucoup de bande passante : la moitié de la bande passante sert à transférer le double du message dans l'unique but de détecter les erreurs. Autrement dit, la bande passante initiale, c'est-à-dire le débit nominal de la liaison, est divisé par deux. Deuxièmement, elle peut ne pas détecter une erreur qui se produit si l'erreur intervient au même endroit dans le message initial et dans son double. Elle peut également indiquer qu'il y a une erreur alors qu'il n'en est rien dans le cas où le message ne subit pas d'erreur mais son double en subit une.

Pour corriger ces problématiques, on utilise des mécanismes plus sophistiqués. Pour le premier problème et donc être plus efficace, plutôt que d'envoyer le message en double, on utilise le **checksum** ou **somme de contrôle**. Il s'agit d'un calcul sur les données avant et après transmission qui va servir d'élément de comparaison. Pour le deuxième problème lié à l'imperfection de la comparaison, il est nécessaire de mettre en

place des mécanismes de détection d'erreurs à plusieurs niveaux dans différents protocoles et avec des calculs de checksum différents également. Cela permet de s'assurer que, même si une erreur n'est pas détectée à un niveau donné, elle le sera à un autre niveau.

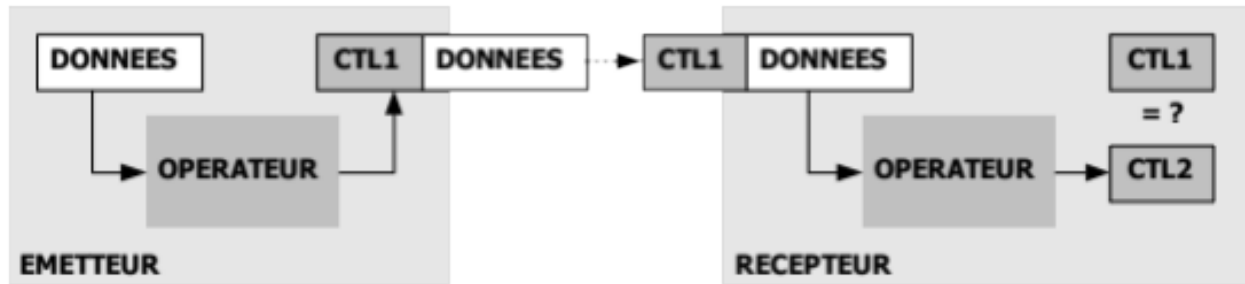


Figure 5: La détection par clé calculée

La somme de contrôle, également connue sous le nom de **clé calculée**, est une séquence binaire générée à partir des données avant leur transmission suite à une opération (un calcul). Elle est calculée sur l'émetteur et envoyée au récepteur avec les données. Sur le récepteur, une nouvelle somme de contrôle est calculée à partir des données reçues. Si les deux sommes de contrôle sont égales, le protocole considère que les données ont été transmises sans erreur. Si les sommes de contrôle diffèrent, il s'agit d'une détection d'erreur, les données devront donc être retransmises.

Il est évident que nous sommes confrontés à la même difficulté, à savoir que lorsqu'une erreur se produit dans la somme de contrôle transmise, le protocole considère qu'il y a une erreur alors qu'il n'y a peut-être pas d'erreur dans les données. Par conséquent, il est possible que les données soient retransmises à tort. A l'inverse, le calcul de la somme de contrôle peut ne pas détecter certaines erreurs car il peut s'avérer insuffisant.

Un exemple de calcul de somme de contrôle très simple, peu coûteux, rapide à exécuter et qui consiste à calculer la somme de contrôle la plus courte possible est le **bit de parité**. Pour calculer la somme de contrôle, l'opérateur prend la somme des bits à un dans les données modulo deux. Le résultat de l'opération et donc la somme de contrôle est constitué d'un seul bit qui vaut zéro si il y a un nombre pair de bits à un et un si il y a un nombre impair de bits à un. On retrouve ici clairement la problématique liée à la qualité du calcul effectué qui fait que l'on ne peut pas détecter toutes erreurs car l'élément de comparaison est insuffisant. Par exemple, si trois bits changent de valeur pendant le transfert, le calcul révélera la présence d'une erreur alors que si seuls deux bits en changent, cela ne sera pas le cas. En effet, si un nombre pair de bits changent de valeur, le bit de parité a la même valeur avant l'émission et après réception.

Cet exemple de calcul de somme de contrôle montre toutes les limites de la détection par clé calculée. Bien que simple et rapide, il n'est pas très efficace pour détecter les erreurs. Dans les réseaux traditionnels, les protocoles calculent des sommes de contrôle à différents niveaux et de différentes manières pour s'assurer de détecter un maximum d'erreurs. Les calculs effectués sont généralement assez performants et produisent des sommes de contrôle sur deux ou quatre octets : par exemple, quatre octets dans la trame Ethernet pour le FCS (Frame Control Sequence) et deux octets au niveau de TCP (Transmission Control Protocol).

1.2.3 Le mode Send&Wait

Nous allons maintenant explorer les mécanismes qui doivent être utilisés pour garantir la fiabilité dans un réseau. Cela consiste à s'assurer que tout ce qui parvient à sa destination correspond exactement à ce que l'émetteur souhaitait envoyer. Pour ce faire, la **somme de contrôle** est nécessaire afin de vérifier qu'il n'y ait pas d'erreurs dans la trame. Mais il faut également vérifier qu'il n'y a pas de perte de données et que les messages arrivent dans l'ordre dans lequel ils ont été envoyés et sans duplications. En cas de perte ou d'erreur, le principe de base pour garantir la fiabilité consiste à **retransmettre** le message ou les messages perdus ou erronés. Il faut donc **conserver sur l'émetteur chaque message envoyé** jusqu'à ce qu'un

accusé de réception indique qu'il est bien arrivé. Cet accusé de réception, ou "acknowledge", signifie "J'ai bien reçu".

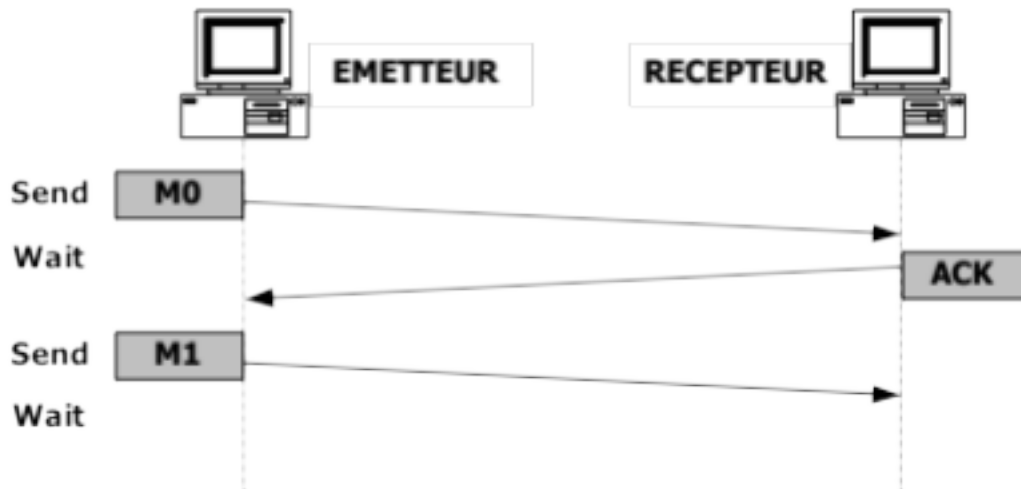


Figure 6: Send and Wait

Pour s'assurer de conserver le message avant de recevoir l'accusé de réception, l'algorithme actuel utilise le mode **send and wait**, c'est-à-dire qu'il envoie un message et attend l'accusé de réception avant d'envoyer le message suivant. Si le message envoyé est perdu ou si une erreur est détectée à la réception via le calcul de la somme de contrôle, le message n'est pas délivré et n'est pas acquitté. Il devra donc être retransmis. La question qui se pose alors est : comment déclencher la retransmission ? Pour cela, l'émetteur déclenche une **temporisation** (*timeout*) pour chaque message envoyé qui spécifie que, par exemple au bout de dix secondes, si l'accusé de réception n'est pas reçu, le message non acquitté doit être renvoyé.

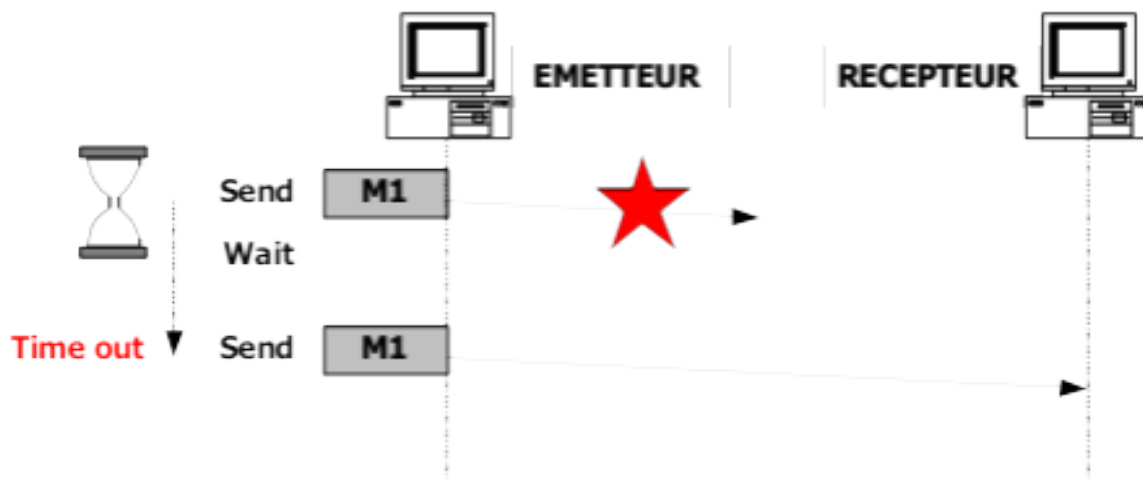


Figure 7: Reprise sur temporisation

Pour l'instant, les mécanismes nécessaires pour garantir la fiabilité sont la somme de contrôle pour détecter les erreurs, l'accusé de réception pour confirmer la réception du message, le stockage du message sur l'émetteur jusqu'à la réception de l'accusé de réception, la retransmission en cas d'erreur ou de perte de données, et le temporisateur de retransmission qui déclenche la retransmission en cas de besoin. Malheureusement, il est possible qu'une retransmission soit déclenchée alors que le message est arrivé correctement. Cela se produit par exemple si le message est bien reçu mais que l'acquiescement est perdu ou qu'il met trop de temps à revenir (plus de temps que la durée du temporisateur). Dans ce cas, le message qui a déjà été bien reçu est

retransmis et donc peut arriver correctement une deuxième fois. Il s'agit d'un double ou d'un duplicata qu'il faut détecter et éliminer.



Figure 8: Perte de l'ACK

Si l'accusé de réception est perdu ou altéré, cela entraîne donc une duplication de message. Si un message est retransmis par erreur alors que le premier message est correctement délivré au système c'est-à-dire transmis aux couches supérieures sur le récepteur, le message en double est éliminé mais l'accusé de réception est envoyé à nouveau afin d'éviter une nouvelle retransmission et supprimer le message qui est stocké sur l'émetteur dans l'attente d'une éventuelle retransmission. Un autre élément important à prendre en compte est que l'accusé de réception peut également arriver trop tard. Si la temporisation de retransmission est mal réglée, c'est-à-dire si elle est trop courte, la retransmission sera déclenchée alors que l'accusé de réception est en train de revenir. Dans ce cas également, le message sera reçu en double alors qu'il n'y avait pas de problème de transmission, simplement l'accusé de réception n'était pas encore arrivé.

Il est important de bien régler la temporisation de retransmission, ni trop courte pour éviter les retransmissions inutiles, ni trop longue pour préserver l'efficacité et ne pas pénaliser le débit utile perçu par l'utilisateur. En cas d'erreur, un temporisateur trop grand est une perte de temps. Idéalement, le temps de retransmission devrait être de l'ordre de quelques RTT (temps aller-retour pour transmettre un bit sur la liaison).

Jusqu'à présent, nous avons introduit les mécanismes suivants pour garantir la fiabilité : la somme de contrôle pour détecter les erreurs, le stockage des messages envoyés non acquittés, l'accusé de réception, la retransmission et la temporisation de retransmission. Il nous faut ajouter la **numérotation des messages** afin non seulement de détecter les duplicata qui peuvent arriver mais aussi de remettre dans l'ordre les messages qui arrivent dans le désordre. En effet, les retransmissions de message peuvent créer du désordre.

Nous introduisons un numéro de message appelé NS (pour "send", ou "envoyé" en français) pour compter les messages envoyés. Ce numéro figurera dans l'en-tête du message et permettra au récepteur de savoir s'il s'agit du message numéro trois ou du message numéro cinq, par exemple. Nous faisons de même pour les messages reçus, en utilisant NR (pour "receive", ou "reçu" en français) pour compter les messages reçus. À quoi sert le numéro de message reçu ? Il sert à comptabiliser les messages qui ont été correctement reçus. Par exemple, si NR vaut 4, cela signifie que l'on attend le message numéro quatre et que l'on a déjà reçu les messages zéro, un, deux et trois. Ce numéro NR nous permet de numéroter les accusés de réception. Pourquoi est-ce utile ? Parce que lorsqu'un message est reçu, afin de détecter les pertes et de garantir l'ordre de réception des messages, il faut vérifier que le numéro du message reçu (NS) correspond au numéro du message attendu (NR). Si tout se passe bien, le numéro du message attendu correspond au numéro du message envoyé. Ainsi, chaque fois qu'un message est reçu, le numéro du message envoyé figure dans l'en-tête du message et doit correspondre au numéro du message attendu qui est stocké sur le récepteur. En d'autres termes, lorsque tout se passe bien, NS est égal à NR en réception.

Maintenant, si **NS (numéro de message envoyé) est inférieur à NR** (numéro de message attendu), cela signifie par exemple que le message numéro deux est reçu alors que le message numéro trois est attendu. Cette situation permet de détecter l'arrivée d'un message en double. Dans cet exemple, le message numéro

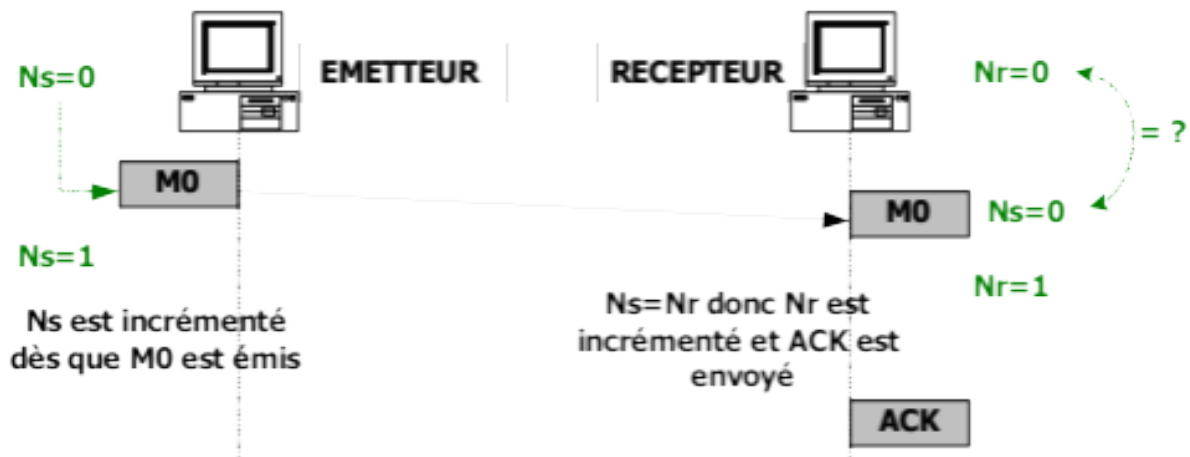


Figure 9: Numérotation des messages émis

deux a déjà été reçu puisque le message numéro trois est attendu. Le récepteur élimine alors le double et renvoie à nouveau l'accusé de réception pour indiquer qu'il attend le message suivant. Le cas **NS supérieur à NR** est également possible : par exemple le message numéro quatre est reçu alors que le message numéro trois est attendu. Cette situation se produit lorsqu'un message a été perdu ou qu'il n'est pas encore arrivé. Il s'agit donc d'un message qui arrive dans le désordre. Dans ce cas, un acquittement est renvoyé indiquant le numéro de message attendu afin de déclencher la retransmission du ou des messages manquants. Dans les deux cas de figure (double ou désordre), il s'agit d'une anomalie que le protocole doit corriger grâce à l'envoi du récepteur à l'émetteur du numéro de message attendu (NR).

Le numéro de message envoyé NS est initialisé à zéro et incrémenté à chaque envoi de message sur l'émetteur. Le compteur NR sur le récepteur est également initialisé à zéro et incrémenté si le message reçu ne contient pas d'erreur et qu'il correspond au message attendu, c'est-à-dire lorsque NS est égal à NR.

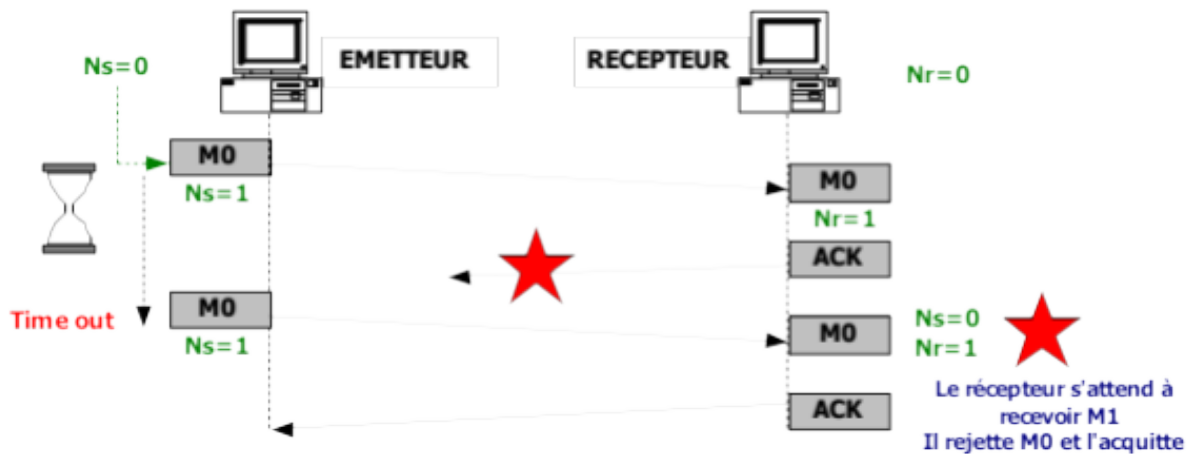


Figure 10: Perte de l'acquittement

Sur le schéma ci-dessus, nous pouvons voir la situation du message qui arrive en double (NS inférieur à NR) : le message numéro zéro est reçu une deuxième fois alors que le message numéro un attendu car l'acquittement a été perdu.

Sur le schéma ci-dessous, la situation NS supérieur à NR se produit : le message numéro deux arrive alors que le message numéro un est attendu. Cet exemple illustre la nécessité de numéroter l'accusé de réception avec NR même dans le mode "Send&Wait" sinon il est possible de croire avoir bien reçu certains messages

alors que ce n'est pas le cas. Sur cet exemple, le message M0 est transmis et bien reçu.

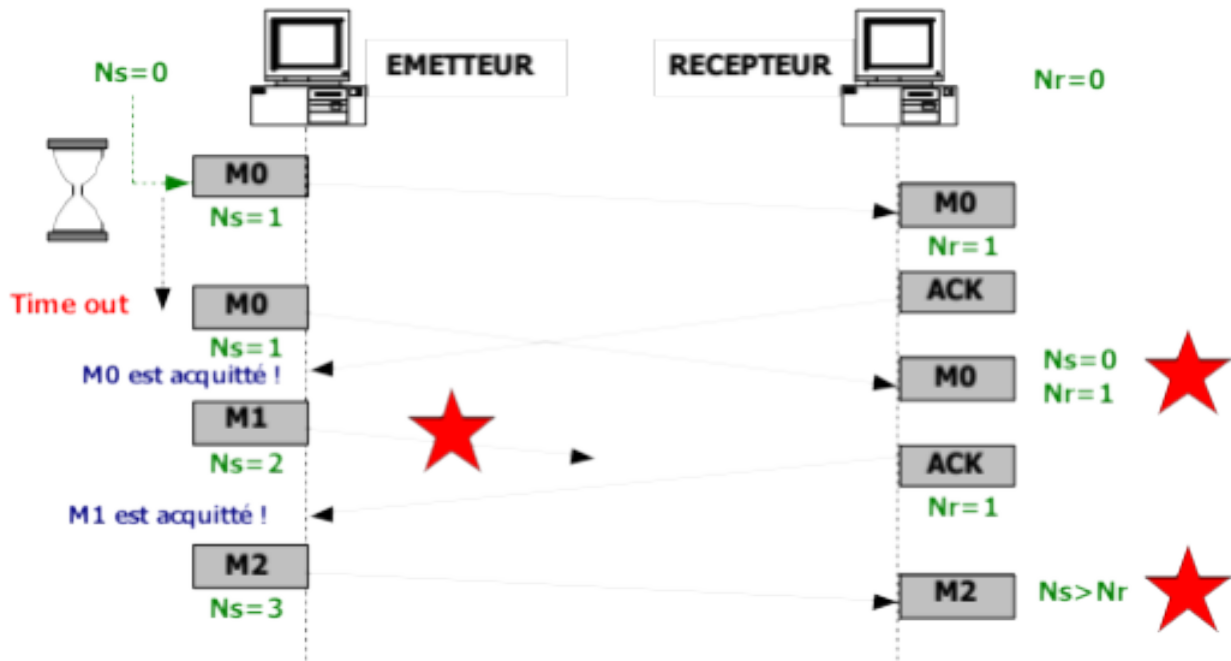


Figure 11: Délai d'accusé trop long

Mais l'accusé de réception met trop de temps à revenir, car le timeout est trop petit et donc la retransmission de M0 se fait avant la réception de l'accusé de réception. M0 va donc arriver en double. Lorsque l'accusé de réception arrive, le message M0 est acquitté et l'émission de M1 devient possible. Lors de la réception du double de M0, le double est détecté grâce à la numérotation des messages reçus. Ce double doit à nouveau être acquitté. Mais si M1 est perdu comme indiqué sur le schéma, l'accusé du double sera considéré comme étant l'accusé de M1 permettant la transmission de M2 qui va arriver avant M1. Cet exemple montre simplement qu'il est nécessaire de numéroté les accusés de réception pour indiquer quel est le numéro du message attendu et donc quels sont les messages qui ont été reçus correctement. Sinon, l'émetteur ne peut pas savoir quel message a été correctement reçu en réception. Par conséquent, l'accusé de réception doit contenir dans son en-tête le numéro NR.

Nous avons donc examiné tous les mécanismes qui sont nécessaires pour garantir la fiabilité. Nous les résumons une dernière fois. Chaque message envoyé doit être mémorisé par l'émetteur afin de pouvoir être retransmis ultérieurement. Nous disposons du délai de retransmission qui active ces retransmissions lorsque le message n'a pas été acquitté. Nous avons donc besoin de la retransmission et de l'accusé. Enfin, la somme de contrôle est nécessaire pour détecter les erreurs et il faut numéroté les messages envoyés (NS) et reçus (NR).

1.2.4 Améliorations du "send and wait"

Nous allons maintenant aborder les mécanismes permettant d'améliorer l'efficacité des algorithmes que nous venons de voir en réduisant le temps consacré à la retransmission de messages. Le premier de ces mécanismes est le **piggybacking**. Mais qu'est-ce que le piggybacking ? Cela consiste à inclure les confirmations (ou accusés) dans des messages contenant des données, ce qui permet d'économiser des confirmations. En pratique, à chaque fois qu'une trame est transmise, le numéro du message envoyé N_s est inclus dans l'en-tête comme nous l'avons vu précédemment. Le piggybacking consiste à ajouter également le numéro du message attendu N_r dans l'en-tête. Ainsi, chaque trame envoyée contenant des données sert aussi de confirmation dans l'autre sens pour l'interlocuteur. Voilà comment le piggybacking permet, lorsque tout se passe bien, de ne jamais envoyer de confirmation explicite. Une **confirmation explicite** est un message spécifiquement

conçu pour réaliser l'acquittement alors qu'il ne contient aucune donnée utile en soi. Il se contente d'un en-tête comportant au minimum le numéro du message attendu, c'est-à-dire NR.

1.2.5 Efficacité d'un protocole

Pour introduire la notion d'efficacité d'un protocole et mesurer l'impact des différentes pertes de temps liées à la mise en œuvre de la fiabilité, il convient de définir ce qu'est l'efficacité en général. En général, l'efficacité mesure ce qui est utile relativement au total de ce qui a été accompli. Dans le cas d'un protocole, les données utiles sont celles qui viennent de la couche supérieure, c'est-à-dire celles qui nécessitent la transmission. Les données non utiles sont celles que le protocole doit ajouter dans l'en-tête ou l'en-queue pour mettre en œuvre son algorithme, ainsi que le temps perdu pour réaliser cet algorithme.

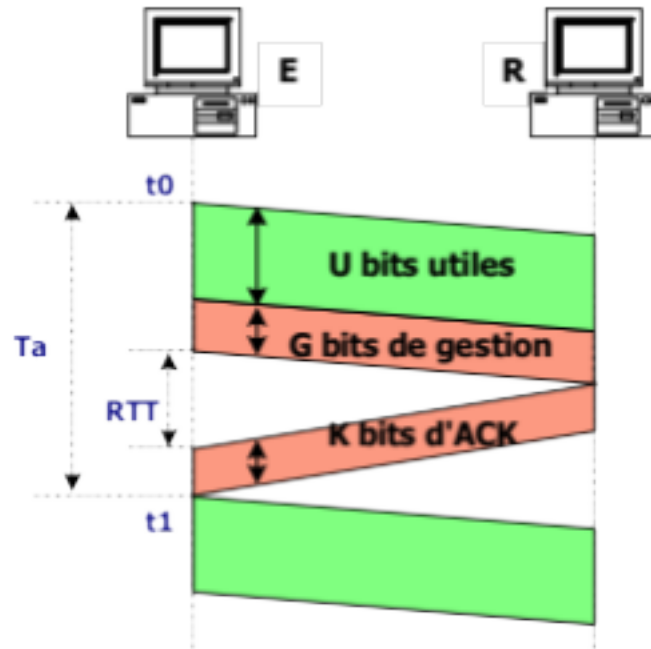


Figure 12: Efficacité d'un protocole

Revenons à notre exemple de transmission d'une trame entre un émetteur et un récepteur. Cette trame contient U bits utiles provenant de la couche supérieure et G bits de gestion, c'est-à-dire la taille de l'en-tête. Pour garantir l'arrivée correcte du message, il est confirmé à sa réception. La confirmation contient K bits non utiles, qui correspondent à un message créé par le protocole lui-même. En général, la taille de la confirmation est égale à celle de l'en-tête, soit $G=K$. Le temps de transmission de la confirmation est le RTT (temps aller-retour).

L'efficacité mesure l'utile par rapport au total et est exprimée en pourcentage. Elle varie toujours entre 0 et 1. Par exemple, si l'efficacité est de 0,8, cela signifie que 80% des bits transmis sont utiles, soit une perte de temps de 20% et donc une chute du débit de 20%. Dans notre exemple, l'efficacité est sans erreur car nous supposons que le message est confirmé immédiatement. Elle s'évalue par le rapport $U/(U+G+K+D \cdot RTT)$, où U est le nombre de bits utiles, G et K sont respectivement les bits de gestion et non utiles ajoutés par le protocole, D le débit, donc $D \cdot RTT$ correspond au temps perdu à attendre la confirmation, exprimé en nombre de bits qui auraient pu être transmis pendant ce temps. Soit N tous les bits réellement transmis jusqu'à la réception de la confirmation : $N=U+G+K$.

Chaque protocole ajoute du temps perdu lié aux traitements réalisés, des en-têtes et des en-queues qui, du point de vue de l'utilisateur, correspondent également à du temps perdu et réduisent donc l'efficacité, c'est-à-dire induisent une baisse du débit. Si vous mesurez le temps nécessaire pour envoyer des données sur une liaison à débit maximal théorique de 1 Gb/s, vous constaterez que le fichier de 1 Gb ne sera pas transmis en

1 seconde en raison de la perte de débit due à tous les protocoles utilisés.

Si on considère une architecture de réseau qui empile plusieurs protocoles, le débit utile sera déterminé par le produit des efficacités de chaque protocole, tel que FTP (Protocole de Transfert de Fichiers), TCP, IP et Ethernet, multiplié par le débit nominal (débit maximum théorique de la liaison). Typiquement, lorsqu'on effectue un transfert de fichiers en utilisant le protocole FTP qui utilise TCP, qui lui-même utilise IP et Ethernet sur la liaison, chaque protocole fait légèrement diminuer le débit du fait des services qu'il rend et de l'algorithme qu'il exécute. Par exemple, le calcul d'un checksum prend un peu de temps et réduit le débit nominal.

Nous distinguons donc le **débit maximum théorique ou débit nominal** qui est le débit initial sur la liaison, du **débit utile ou débit réel**, qui représente le débit réellement observable par l'utilisateur du réseau par exemple en mesurant avec sa montre le temps de téléchargement d'un fichier.

Pour intégrer les erreurs dans le calcul de l'efficacité, nous pouvons considérer la probabilité de recevoir correctement une trame, qui est $(1-BER)^N$, où BER est le taux d'erreur binaire et N est le nombre de bits transmis, comprenant les bits utiles, l'acquittement, ainsi que les en-têtes et en-queues. Pour intégrer cela dans l'efficacité, il suffit de multiplier l'efficacité sans erreur par cette probabilité de trame correcte. En d'autres termes, l'efficacité diminuera lorsque la probabilité d'erreurs augmente. Par exemple, si la probabilité de trame erronée est de 20%, cela entraînera une baisse de débit de 20%.

En effet, il y a un compromis à trouver entre la taille des trames et l'efficacité dans le cas où il y a des erreurs. Plus une trame est grande, plus elle est sujette aux erreurs, et il est donc avantageux de la rendre plus petite. Cependant, dans le cas où il n'y a pas d'erreurs, l'efficacité est maximisée lorsque la trame contient un grand nombre de données utiles par rapport au temps perdu avec les en-têtes et les checksums. Il est donc important de trouver un compromis en fonction du taux d'erreur binaire pour atteindre l'efficacité maximale.

1.2.6 Protocoles à anticipation

Pour poursuivre notre réflexion sur l'amélioration de l'efficacité, nous avons évoqué précédemment la technique du Piggybacking, qui permet de réaliser des économies sur les acquittements explicites. A présent, nous allons aborder l'**anticipation**.

L'anticipation consiste à envoyer plusieurs messages de manière consécutive, sans attendre la réception des accusés de réception des messages précédents. Cela permet de gagner du temps en évitant les délais liés à la transmission des accusés de réception (fonction du RTT). Cependant, cette technique nécessite une **fenêtre d'émission suffisamment grande pour stocker tous les messages envoyés non encore confirmés**. Quand la fenêtre d'émission est pleine, les émissions deviennent impossibles et l'anticipation s'arrête. À chaque réception d'un accusé de réception, les messages acquittés peuvent sortir de la fenêtre d'émission libérant ainsi de la place l'envoi de nouveaux messages. L'anticipation améliore l'efficacité du protocole car elle permet d'envoyer des messages sans attendre l'acquittement des messages précédemment envoyés. La capacité d'anticipation dépend de la taille de la fenêtre d'émission.

Quelle est la taille optimale de la fenêtre d'émission ? Bien sûr, plus elle est grande, mieux c'est. Mais il est également important d'économiser l'espace mémoire. En effet, les cartes réseau ne disposent pas d'un espace mémoire gigantesque. De même si de nombreuses applications utilisent Internet et qu'elles demandent chacune une grande quantité de mémoire pour pouvoir faire l'envoi des messages par anticipation, les limites du système d'exploitation peuvent être rapidement atteintes. Il est donc important de connaître la taille optimale de la fenêtre d'émission pour pouvoir anticiper au maximum sans jamais bloquer l'émetteur mais tout en économisant la mémoire. Cette taille optimale dépend du RTT et du débit d'émission. En effet, pour ne jamais bloquer l'émetteur il faut qu'il puisse envoyer des messages pendant le temps d'attente de la confirmation c'est-à-dire pendant le temps RTT. Autrement dit, si un message est émis pendant une seconde et que le RTT est de dix secondes, il faudra une fenêtre d'émission de taille 10, car il faut pouvoir émettre neuf messages pendant le RTT, qui dure donc environ neuf secondes. Le temps d'émission de chaque message dépend du débit car il est égal à la taille du message divisé par le débit. Par conséquent, la taille optimale de la fenêtre d'émission dépend en grande partie du RTT, mais également de la taille maximale de la trame

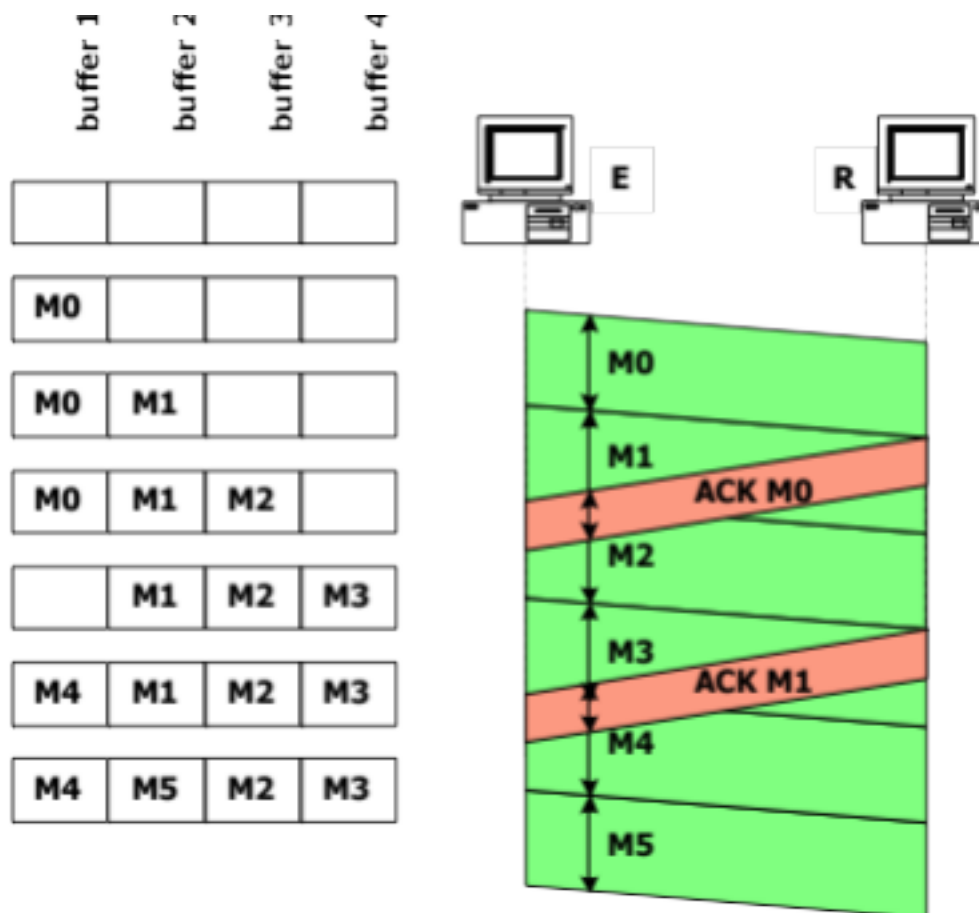


Figure 13: Protocole à anticipation

et du temps qu'il faut pour l'émettre, c'est-à-dire du débit. Plus le RTT est élevé, plus la fenêtre d'émission doit être grande. Plus le débit est élevé et plus le temps d'émission sera court, ce qui implique également une fenêtre d'émission de grande taille.

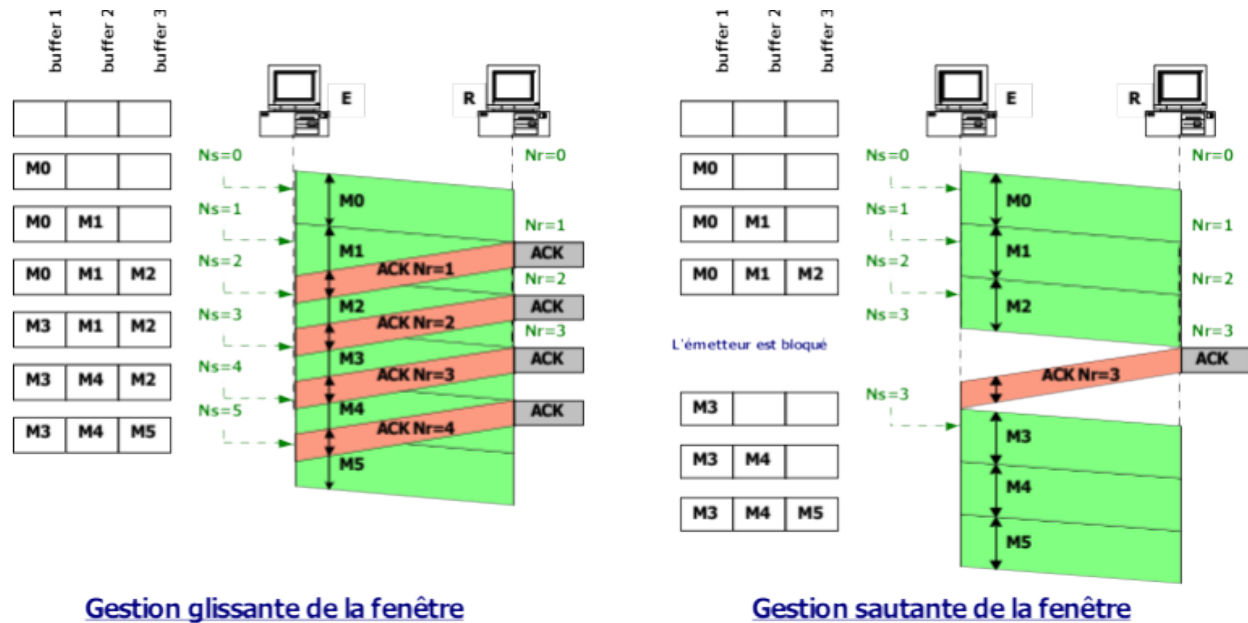


Figure 14: Fenêtre glissante et fenêtre sautante

Un autre mécanisme permettant d'améliorer l'efficacité est la **gestion sautante de la fenêtre**. L'objectif est toujours d'économiser les acquittements en ne confirmant pas chaque message dès sa réception. Par exemple, TCP ne confirme que tous les trois messages. Ainsi, le protocole attend d'avoir reçu M0, puis M1, puis M2 avant d'envoyer ACK NR=3. Dans l'en-tête, il y a le numéro du message attendu NR et l'émetteur qui reçoit la confirmation saura que les trois messages précédents ont bien été confirmés et qu'il peut vider sa fenêtre. Si la fenêtre peut contenir au maximum trois messages et que l'on ne confirme que tous les trois messages, on envoie M0, M1 et M2, puis on est bloqué jusqu'à ce que l'on reçoive la confirmation ACK NR=3. On envoie alors M3, M4 et M5, puis on est de nouveau bloqué jusqu'à ce que l'on reçoive la confirmation ACK NR=6. Et ainsi de suite. C'est pourquoi on parle de gestion sautante de la fenêtre. À l'inverse, si chaque message reçu est confirmé aussitôt, on parle de **gestion glissante de la fenêtre**. On envoie M0, puis M1 et M2, mais avant d'envoyer M2, on reçoit la confirmation de M0 qui retire M0 de la fenêtre d'émission permettant ainsi l'envoi de M3 après M2. Et ainsi de suite. La gestion glissante de la fenêtre est donc plutôt comme un tourniquet.

Le plus efficace est la gestion sautante de la fenêtre, mais avec une fenêtre suffisamment grande pour ne pas être bloqué en émission. Cela dépend donc de la taille optimale de la fenêtre. Même si l'on ne confirme que tous les trois messages, il faut être en mesure de continuer à anticiper et donc de ne pas perdre de temps dans l'attente de l'acquiescement. Dans l'exemple ci-dessus, il faut donc une fenêtre plus grande que trois messages pour être plus efficace.

Pour terminer, un dernier mécanisme d'optimisation de l'efficacité concerne les messages reçus dans le désordre. Si l'on attend par exemple M3 et qu'on reçoit M4, que faire du message M4 ? En général, les protocoles de liaison font du **rejet simple**, c'est-à-dire que tous les messages reçus dans le désordre sont rejetés dans l'attente de la retransmission du ou des messages manquants et de tous les suivants. Si l'on reçoit un message qui n'est pas celui attendu, on demande la retransmission de tous les messages à partir du message manquant. Cela n'est pas très efficace car génère beaucoup de retransmissions.

L'optimisation consiste à ne retransmettre que les messages manquants et donc à conserver sur le récepteur les messages reçus dans le désordre. Il s'agit du **rejet sélectif**. Pour ce faire, on introduit une fenêtre de

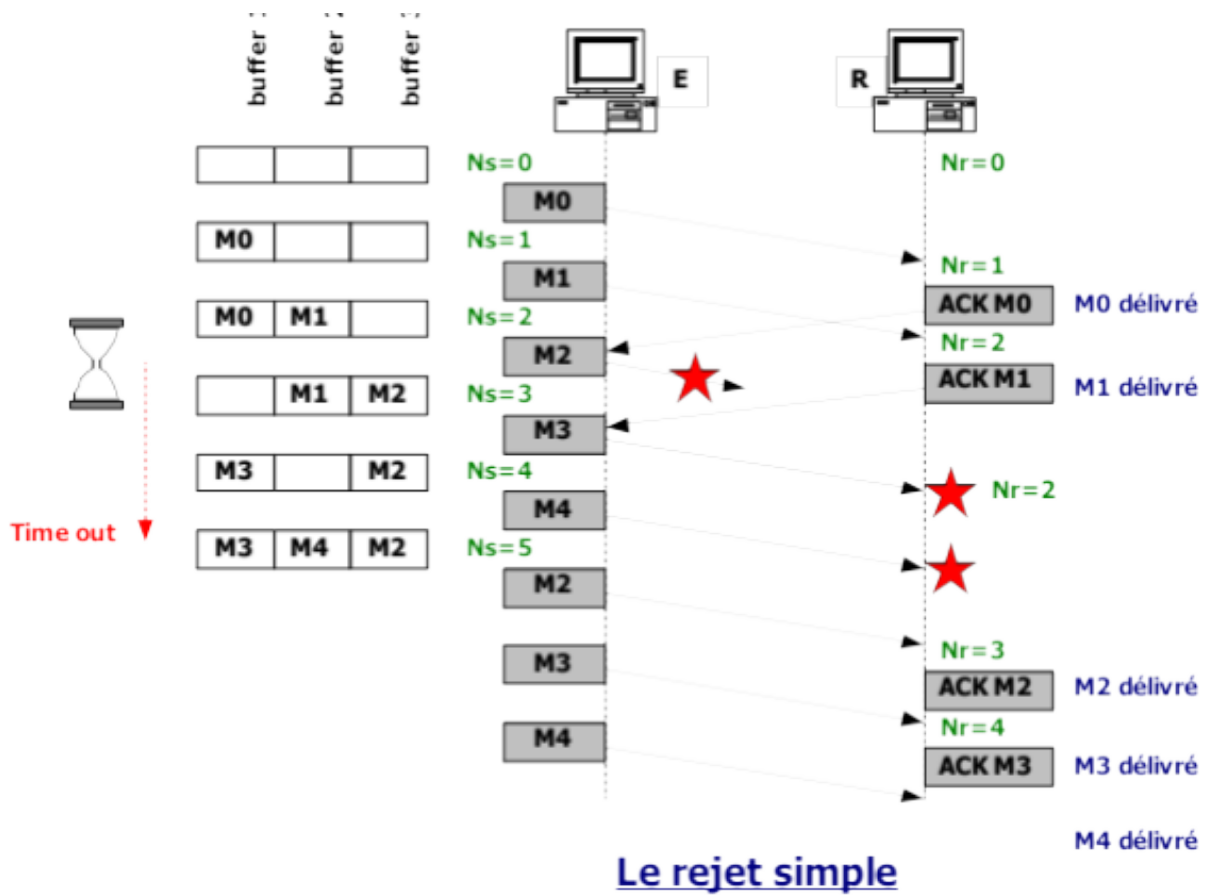


Figure 15: Rejet simple

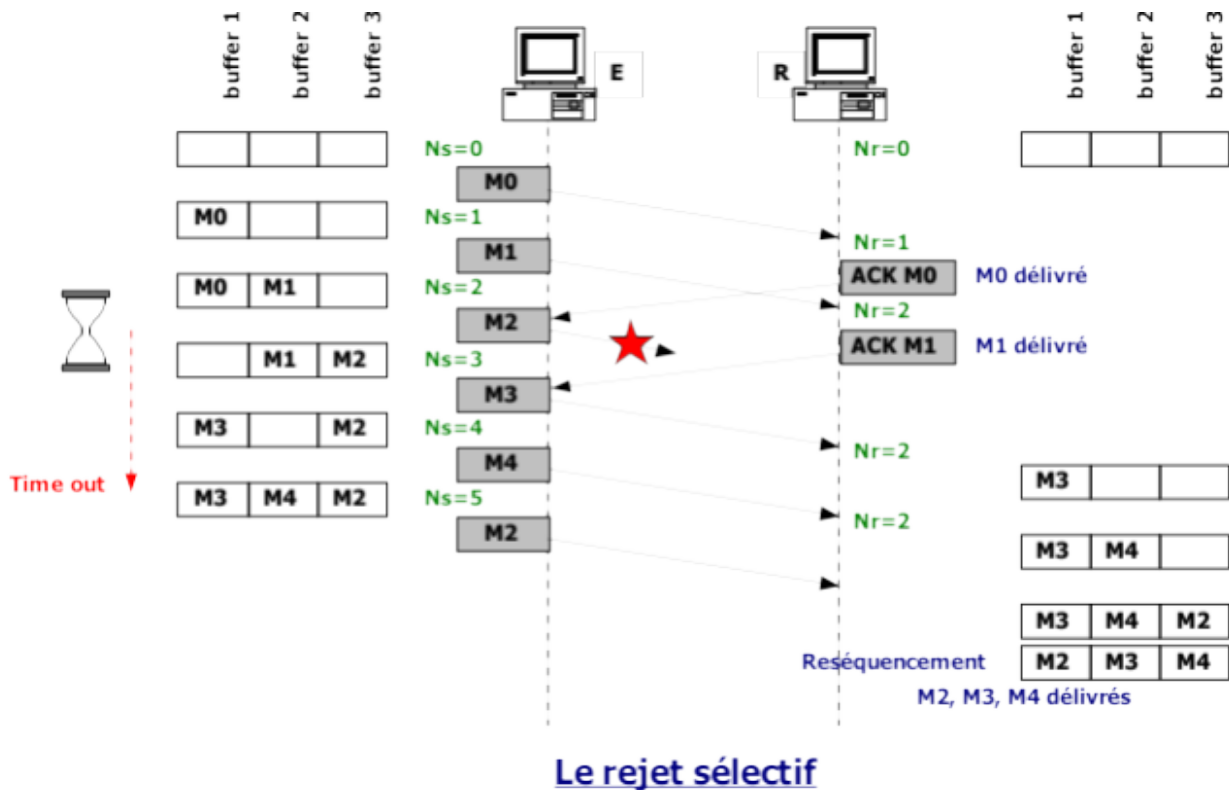


Figure 16: Rejet sélectif

réception pour stocker les messages reçus dans le désordre. Si l'on reçoit par exemple M3 et M4 alors que l'on attend encore M2, on peut stocker M3 et M4 dans la fenêtre de réception et demander la retransmission de M2. Lorsque M2 arrive, on peut remettre tous les messages dans l'ordre et vider la fenêtre de réception pour transmettre les messages à la couche supérieure. Le rejet sélectif permet d'éviter des retransmissions, mais nécessite suffisamment de mémoire pour stocker les messages reçus dans le désordre. Il est généralement utilisé sur des liaisons où les retransmissions sont coûteuses, en particulier sur les réseaux longue distance (ayant un grand RTT) tels que les liaisons satellites. En revanche, sur des liaisons de type réseaux locaux Ethernet, on utilise plutôt le rejet simple car les cartes réseau n'ont généralement pas beaucoup de mémoire pour mettre en œuvre le rejet sélectif. De plus, la fiabilité est assurée par TCP plutôt que par la carte réseau.

Pour récapituler, la fenêtre d'émission stocke tous les messages envoyés qui n'ont pas encore été confirmés. Elle permet de faire de l'anticipation et se remplit à chaque envoi de message et se vide à chaque confirmation. Sa taille optimale dépend du RTT et du débit et vise à éviter d'être bloqué en émission en attendant les confirmations qui n'ont pas encore été reçues. La fenêtre de réception, utilisée lors du rejet sélectif, stocke tous les messages reçus dans le désordre. Elle se remplit lorsque l'on reçoit un message qui n'est pas celui attendu et se vide lorsque l'on a reçu tous les messages manquants et qu'on a pu les remettre dans l'ordre. Pour améliorer l'efficacité, nous avons vu le Piggybacking, qui consiste à envoyer le numéro du message attendu dans l'entête de chaque trame contenant des données et sert à effectuer des confirmations lors de l'envoi de données. Nous avons également vu la gestion sautante de la fenêtre, qui permet d'économiser des acquittements en confirmant moins fréquemment chaque message reçu. Tous ces mécanismes permettent d'être plus efficace.

1.2.7 Contrôle de flux

Notre dernier mécanisme visant à accroître l'efficacité est le **contrôle de flux**. Le contrôle de flux nous permet de minimiser les pertes en agissant de la manière suivante : lorsque le récepteur est saturé c'est-à-dire

qu'il ne peut plus recevoir de données, afin d'éviter de les perdre, il prévient l'émetteur et lui demande de "s'arrêter". Cela peut ressembler à un "go ! stop ! go ! stop !". Le contrôle de flux permet donc au récepteur de ralentir l'émetteur dans ses émissions afin d'éviter les pertes. Formellement, le contrôle de flux est le mécanisme consistant à asservir le rythme d'émission de l'émetteur aux capacités de réception du récepteur. Donc c'est soit "go"/"stop", c'est-à-dire envoyer ou arrêter d'envoyer. Ou cela peut être plus subtil : lorsque le récepteur signale à l'émetteur dans l'en-tête de chacun de ses messages envoyés combien d'octets il est prêt à recevoir à cet instant. C'est ainsi que fonctionne le contrôle de flux dans le protocole TCP.

Nous avons donc examiné tous les mécanismes permettant de garantir la fiabilité avec les algorithmes associés. Nous avons vu comment mesurer l'efficacité et comment elle peut être améliorée grâce à de nouveaux mécanismes. Enfin, nous avons abordé le débit utile, qui permet de mesurer le temps perdu dans l'exécution des algorithmes de chaque protocole.

1.3 La signalisation

La signalisation dans les réseaux est tout ce qui est transmis dans le réseau qui ne correspond pas aux données utiles, mais qui est nécessaire pour le bon fonctionnement du réseau. Cela inclut donc toutes les en-têtes, toutes les en-queues, le temps perdu et les messages explicitement conçus pour faire fonctionner le réseau. Par exemple, lorsque vous mesurez le temps d'une conversation téléphonique ou la quantité de données transférées sur Internet depuis votre smartphone, cela inclut également les messages transmis dans le réseau qui servent à informer votre fournisseur d'accès de votre consommation et qui sont nécessaires au fonctionnement du réseau, mais qui ne sont pas utiles pour l'utilisateur. C'est cela la signalisation.

1.4 Le protocole HDLC

Nous allons maintenant examiner le protocole HDLC, qui illustre comment la fiabilité est mise en œuvre de manière concrète sur une liaison dans un protocole.

Le protocole HDLC est un protocole de liaison (niveau deux dans le modèle OSI) permettant la communication de trames entre deux cartes réseau. Conçu en 1976, il est encore utilisé dans divers réseaux, tels que les réseaux mobiles, et des variantes de HDLC existent également. Par exemple, le protocole LLC qui normalise les réseaux locaux s'en inspire grandement ainsi que certains protocoles propriétaires Cisco, l'un des plus grands fabricants d'équipements réseaux.

Le protocole HDLC a été conçu pour fonctionner sur plusieurs types de liaisons, telles que les liaisons point à point symétriques ou dissymétriques et les liaisons multipoints dissymétriques. Cela signifie que lorsqu'on utilise une liaison multipoint, il est nécessaire d'ajouter l'adresse du destinataire dans l'en-tête de la trame. Ce protocole a donc été conçu pour fonctionner sur ces différents types de liaisons point à point ou multipoint. Cela implique également que, lors de l'ouverture de la liaison, lorsque les cartes réseau de chaque côté de la liaison sont activées, elles doivent s'accorder sur le type de liaison à utiliser. Pour chaque type de liaison, un message spécifique indiquant le mode d'ouverture de la liaison sera nécessaire. Par exemple, la trame SNRM indique que la liaison est ouverte dans le mode Normal Response Mode, ce qui correspond à une liaison multipoint dissymétrique. On retrouve également les modes ABM ou ARM pour les autres types de liaisons prévus dans le protocole.

Lorsqu'une trame est transférée sur une liaison, chaque trame est délimitée par un fanion pour indiquer le début et la fin de la trame. Un checksum est utilisé en fin de trame sur deux octets pour s'assurer qu'il n'y a pas d'erreur dans la trame. Ce protocole utilise également des mécanismes pour améliorer l'efficacité, tels que le contrôle de flux, le piggybacking et l'anticipation.

Le protocole HDLC définit trois types de trames :

- **Les trames I**, qui contiennent des données et doivent donc être numérotées en émission avec NS.
- **Les trames S**, qui servent à superviser l'échange, c'est-à-dire à envoyer des acquittements, des rejets et à contrôler le flux et doivent donc contenir le numéro de trame attendue NR.
- **Les trames U**, qui sont non numérotées, c'est-à-dire qu'elles ne contiennent ni NS ni NR, et qui permettent l'ouverture et la fermeture de la liaison.

Le format de la trame HDLC comprend donc un octet pour l'adresse destinataire, un octet pour le champ de commande qui définit le type de trame et contient les numéros NS et NR selon le type de trame, et deux octets pour le checksum. La transparence binaire est assurée grâce à la technique du bit de bourrage, qui permet de dissocier les fanions qui pourraient se trouver à l'intérieur de la trame.

Parmi les différents types de trames S, quatre sous-types sont définis pour les acquittements, qui contiennent le numéro NR (numérotation des acquittements) :

- **RR** (Receive Ready) est une trame d'acquiescement standard. Elle est utilisée lorsqu'un acquiescement explicite est nécessaire.
- **RNR** (Receive Not Ready) est utilisée pour le contrôle de flux. Lorsqu'un équipement envoie cette trame, elle sert d'acquiescement mais indique également à l'émetteur d'arrêter d'envoyer.
- Les trames **REJ** (Reject) et **SREJ** (Selective Reject) permettent de signaler qu'une trame a été reçue dans le désordre : REJ signifie "Reject" (rejet simple), tandis que SREJ signifie "Selective Reject" (rejet sélectif).

Ces quatre trames RR, RNR, REJ et SREJ sont des trames de supervision qui ont chacune leur signification et qui contiennent toutes le numéro NR, c'est-à-dire le numéro de message attendu, et qui permettent d'acquiescer.

Le champ de commande, qui occupe un octet, comprend huit bits et permet de distinguer tous les types de trames que nous avons vus et de stocker les numéros de messages. La trame I est reconnue lorsque le bit zéro (poids faible) contient zéro. Les trames S et U contiennent un sur le bit zéro et sont distinguées sur le bit un par une valeur à zéro ou à un. Chaque trame stocke soit NS, soit NR, soit les deux soit aucun des deux. Les compteurs NS et NR sont sur trois bits, ce qui signifie qu'ils peuvent varier de 0 à 7. Cela permet d'envoyer plusieurs messages à la suite. NS est stocké uniquement dans la trame I, car c'est la seule trame qui contient des données devant être stockées pour une éventuelle retransmission et qui a donc besoin de numéroté les messages envoyés. NR est également stocké dans la trame I, ce qui permet de mettre en œuvre le piggybacking. NR est également présent dans les trames S, car ce sont les trames d'acquiescement. Ces trames ne contiennent pas NS car elles ne transportent pas de données. Pour distinguer les quatre sous-types de trame S, nous avons besoin de deux bits, car il faut pouvoir coder quatre trames différentes sur deux bits. Les bits deux et trois permettent de le faire. Pour les trames U, qui ne contiennent ni NS ni NR, cinq bits permettent de distinguer les différents types de trame U. Le bit 4 contient le bit P ou F, qui permet de signaler la présence d'un problème. Par défaut, ce bit est à zéro lorsque tout se passe bien. S'il est positionné à un, cela signifie qu'une réponse de l'autre partie est attendue. Typiquement, lorsqu'un émetteur envoie plusieurs trames à la suite et que sa fenêtre d'émission est pleine, ces émissions sont bloquées. Un acquiescement explicite est donc nécessaire pour libérer de la place dans la fenêtre et poursuivre les émissions. Pour demander l'acquiescement, l'émetteur bloqué positionne ce bit à un. Le récepteur doit alors répondre immédiatement en positionnant à son tour ce bit à un pour indiquer sa réponse.

Nous avons donc parcouru tous les éléments du champ de commande permettant de distinguer les différents types de trame. Ainsi, nous récapitulons ici toutes les valeurs possibles pour ce champ de commande, qui constitue l'en-tête avec l'adresse du destinataire et qui permet, par conséquent, de garantir la fiabilité en ajoutant un octet à l'en-tête (la commande) et deux octets à l'en-queue (la somme de contrôle appelée FCS pour Frame Control Sequence).

En ce qui concerne les ouvertures de liaison, le principe est celui du mode connecté, c'est-à-dire qu'une carte réseau souhaitant ouvrir une liaison pour communiquer avec une autre carte réseau envoie une requête pour l'ouverture dans l'un des modes possibles. Et les autres cartes réseau (si la liaison est multipoint) répondent "d'accord" (UA, un acquiescement non numéroté) ou "pas d'accord" (avec DM pour Disconnect Mode) pour indiquer qu'elles ne sont pas prêtes à se connecter sur cette liaison.

De même, lors de la déconnexion, il y a d'abord une phase de requêtes : une des cartes réseaux demande la déconnexion et l'autre carte réseau accepte ou, au contraire, refuse la déconnexion pour éventuellement terminer des échanges.

Les algorithmes utilisés sont ceux que nous avons vus pour la fiabilité. Nous allons les résumer rapidement. Tout d'abord, le contrôle de flux qui permet d'éviter les pertes ou une station ne pouvant plus recevoir de

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	
Nr			P/F	Ns			0	trame I
Nr			P/F	0	0	0	1	RR
Nr			P/F	0	1	0	1	RNR
Nr			P/F	1	0	0	1	REJ
Nr			P/F	1	1	0	1	SREJ
0	0	0	P	1	1	1	1	SARM
1	0	0	P	0	0	1	1	SNRM
0	0	1	P	1	1	1	1	SABM
0	1	1	P	1	1	1	1	SABME
0	1	0	P	0	0	1	1	DISC
0	1	1	F	0	0	1	1	UA
1	0	0	F	0	1	1	1	CMDR/FRMR
0	0	0	F	1	1	1	1	DM

Figure 17: Récapitulatif des principales commandes

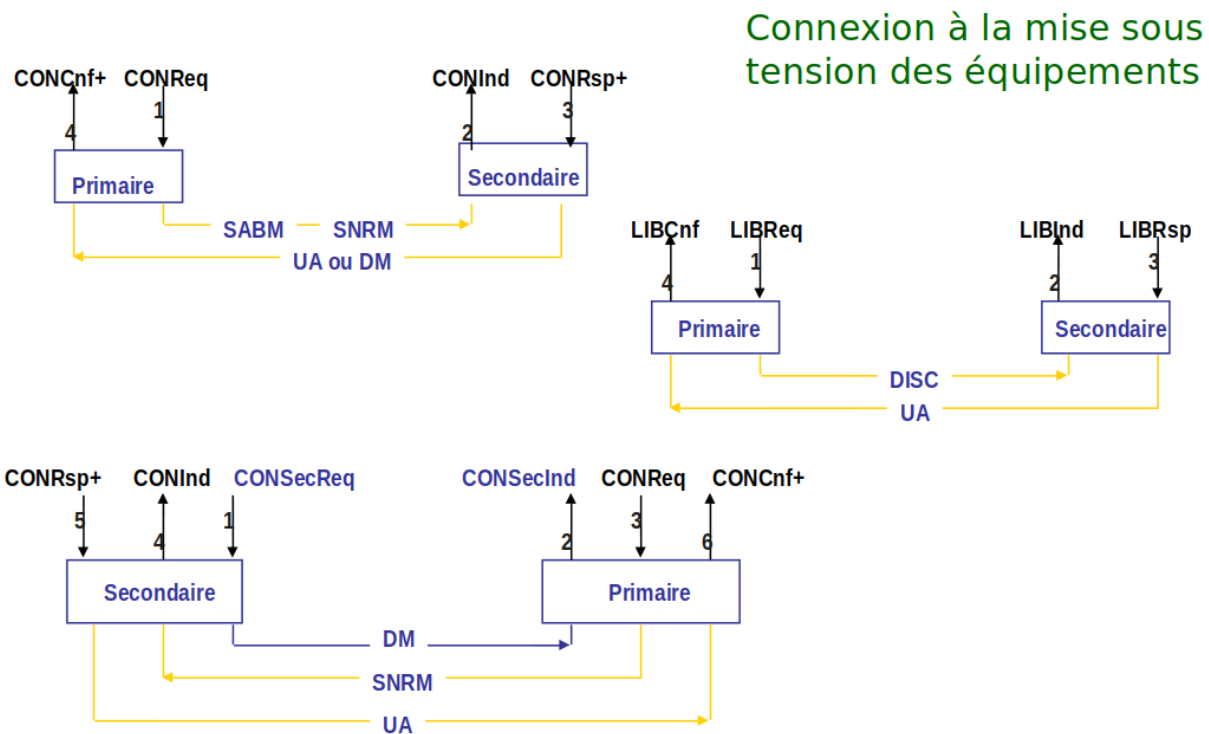


Figure 18: Connexion et libération dans HDLC

trames envoie RNR et, dès qu'elle est à nouveau en mesure de recevoir, elle envoie RR.

En cas d'erreur pendant la transmission d'une trame détectée par le checksum (FCS), la carte réseau détectant une erreur dans le calcul du checksum jette la trame sans envoyer de message, ce qui se traduit pour l'émetteur par une retransmission après le délai de retransmission, comme si la trame avait été perdue.

En cas de réception d'une trame qui n'est pas celle attendue, c'est-à-dire que le numéro de séquence (NS) dans l'en-tête est différent du numéro de trame attendue en réception, le protocole déclenche soit le rejet simple, soit le rejet sélectif selon le réglage de la liaison.

Un temporisateur de retransmission nommé T1 déclenche les retransmissions à l'expiration du délai de temporisation. Ce temporisateur est déclenché pour chaque trame I émise. Un autre temporisateur nommé T2 permet de déclencher l'envoi d'une trame S si aucun acquittement n'a été transmis depuis trop longtemps. Il est nécessaire car le protocole HDLC utilise la gestion sautante de la fenêtre pour plus d'efficacité et donc n'acquiesce pas de manière systématique. Le temporisateur d'acquiescement permet de garantir l'envoi d'un acquiescement au moins tous les temps T2.

1.4.1 Règles de reprise

Examinons maintenant quelques exemples d'échange de données sur une liaison HDLC. Nous avons ici A et B qui sont deux cartes réseau qui s'échangent plusieurs trames contenant de l'information et une fenêtre (W pour Window) de taille 7, qui est la taille maximale de la fenêtre sur une liaison HDLC dans laquelle les compteurs NS et NR sont codés sur 3 bits. En effet, étant donné que les compteurs vont de 0 à 7, il est impossible de stocker dans la fenêtre deux trames ayant le même numéro. C'est pourquoi la taille maximale de la fenêtre est 7, afin de pouvoir y placer les trames de 0 à 6. Il est également impossible de mettre la trame numéro 7 dans la fenêtre, ce qui explique que la fenêtre ne peut pas faire 8, car cela entraînerait une incertitude. En effet, si la fenêtre contient les trames de 0 à 7 et que toutes les trames sont bien reçues, le récepteur attendra la trame 0 suivante. Cependant, si aucune trame n'est bien reçue, le récepteur attendra

toujours la trame 0, mais la première trame 0 de la fenêtre. Si la fenêtre fait 8, il y a une incertitude quant à la façon dont l'émetteur va interpréter le message "j'attends la trame 0".

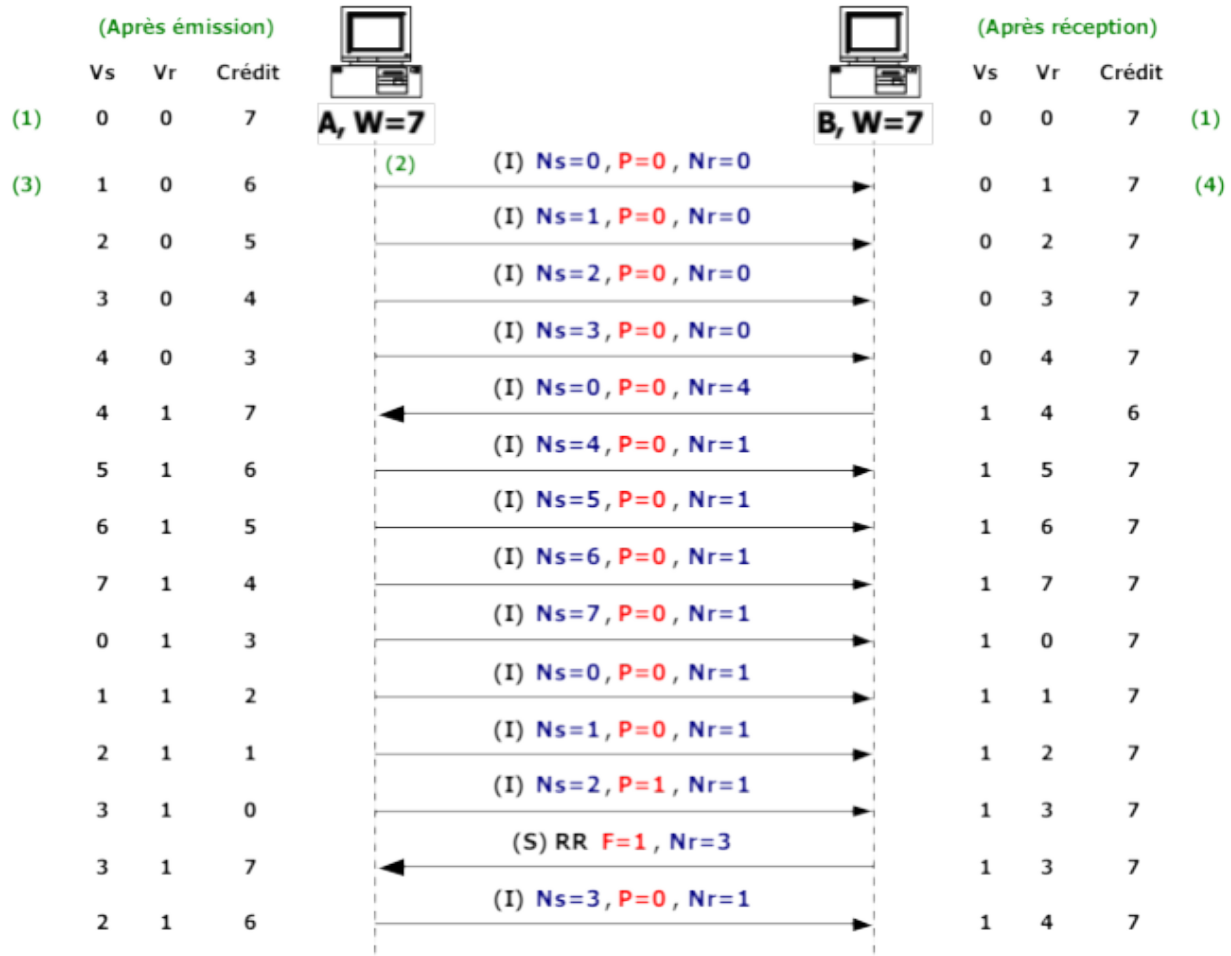


Figure 19: Exemple d'échange de données en HDLC

Dans cet exemple, nous avons deux cartes réseau A et B avec chacune une fenêtre d'émission de taille 7, qui est la taille maximale. A envoie d'abord 4 trames I à B, ce qui provoque l'incrément du compteur VS, qui compte les trames I émises par A, du compteur VR, qui compte les trames I reçues, et des crédits, qui indiquent le nombre de places disponibles dans la fenêtre d'émission. Au départ, les compteurs sont à zéro et les crédits sont à 7. Lorsque A envoie 4 trames I consécutives, le compteur VS est incrémenté et passe de 0 à 4, tandis que les crédits diminuent de 7 à 3, car les 4 trames sont stockées dans la fenêtre d'émission. En réception, à chaque fois qu'une trame I est reçue, le compteur VR augmente et passe de 0 à 4, tandis que VS et les crédits ne changent pas, car nous sommes en réception. Pour que le compteur VR augmente, il y a deux conditions à remplir : la trame est correcte après vérification du checksum et la trame reçue correspond bien à la trame attendue (NS présent dans l'en-tête de la trame reçue = VR le numéro de la trame attendue localement).

Ensuite, B envoie une trame I à A. Cette trame I, avec $NS=0$ et $NR=4$, illustre le Piggybacking, car elle permet d'envoyer la première trame de B vers A mais sert également d'acquittement des 4 trames reçues par B précédemment. Comme cette trame I contient $NR=4$ dans son en-tête, B signifie à A qu'elle a bien reçu les trames I0, I1, I2 et I3 provenant de A. Cette trame I est bien reçue par A, car elle est la trame 0 attendue par A. VR passe donc à 1 sur A et les crédits repassent à 7 l'acquittement des 4 trames précédentes libère I0, I1, I2 et I3 de la fenêtre qui devient à nouveau vide. A continue en envoyant 7 trames I consécutives de I4 à I7 puis de I0 à I2, ce qui remplit complètement la fenêtre et fait donc baisser les crédits à 0. Les trames

envoyées sont les trames numéro 4, puis 5, puis 6, puis 7, puis 0, puisque les compteurs sont sur 3 bits et repassent à 0 après 7, puis 1 et 2.

Comme la fenêtre d'émission est pleine, lors de l'émission de la dernière trame (I2), le bit P est positionné à 1, signifiant que A a besoin d'un acquittement, car elle ne peut plus envoyer de trames tant que la fenêtre d'émission est pleine. À ce moment, B a reçu les 7 trames précédentes et a donc augmenté VR, sachant que la première trame I reçue était aussi un acquittement (NR=1 dans la trame I précédente envoyée par B à A). Lorsque B reçoit la dernière trame I avec le bit P=1, il doit envoyer un acquittement, ce qu'il fait en envoyant RR avec F=1 et NR=3 (il attend la trame 3). Cette trame est donc bien un acquittement des 7 trames I précédemment reçues et permet de vider la fenêtre. Les crédits reviennent à 7 sur A qui peut continuer à envoyer de nouvelles trames I.

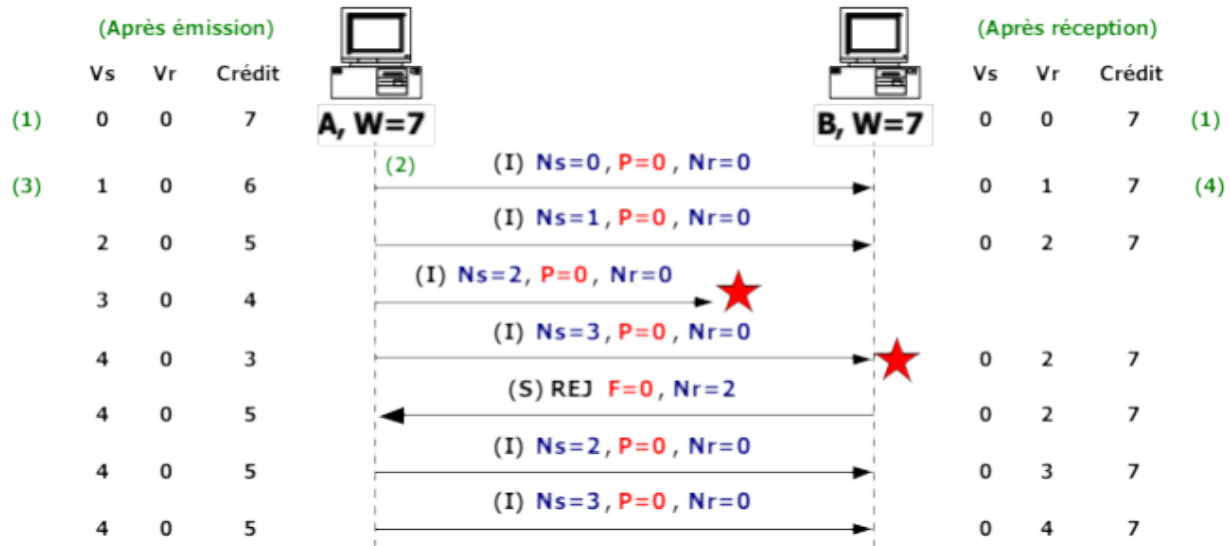


Figure 20: Exemple de gestion d'erreurs

Cet autre exemple permet d'illustrer le rejet simple (REJ). Cela se produit lorsqu'une trame est perdue ou erronée. Par exemple, A envoie la trame zéro (I0,0), puis la trame un (I1,0) et la trame deux (I2,0). Mais la trame deux n'est pas reçue correctement. La trame trois (I3,0) est alors envoyée, car on est toujours dans l'anticipation. C'est à ce moment que B s'aperçoit que la trame reçue n'est pas celle attendue, étant donné que la trame deux n'a pas été reçue correctement. Lorsque la trame 3 arrive, B attend la trame deux donc envoie un rejet REJ avec NR=2 pour indiquer qu'il attend la trame deux et demander la retransmission de toutes les trames à partir de la trame deux. À la réception de cette trame, A va donc acquitter les trames zéro et un, libérant ainsi la fenêtre d'émission en retirant ces deux messages. Cela explique pourquoi les crédits passent de 3 à 5. En même temps, étant donné qu'il s'agit d'un REJ, A déclenche immédiatement, sans attendre le timeout, la retransmission de la trame deux et de la trame 3, car avec le rejet simple, tout est renvoyé à partir de la trame perdue.

Examinons un exemple de contrôle de flux : le récepteur B, après avoir reçu les trames zéro, un et deux, est bloqué car il ne peut plus recevoir (par exemple, en raison de la surcharge des couches supérieures). Dans ce cas, B déclenche le contrôle de flux en envoyant RNR NR=2, ce qui permet d'acquitter les trames zéro, un et deux et de bloquer les émissions sur A. Cependant, A a encore des trames à envoyer. Ainsi, tous les temps T1, il envoie RR P=1 pour demander de débloquer la transmission. Lorsque B sera de nouveau prête à recevoir, elle enverra RR F=1 NR=2 pour indiquer qu'elle est de nouveau prête à accepter des trames et qu'elle attend la trame numéro deux. Ainsi, A pourra reprendre ses émissions.

HDLC met en pratique une gestion sautante de la fenêtre. Du fait du piggybacking et du temporisateur d'acquiescement T2, il est plus efficace de ne pas acquiescer chaque message individuellement. Le protocole fait le choix d'envoyer un acquiescement explicite tous les trois messages si le piggybacking n'a pas permis

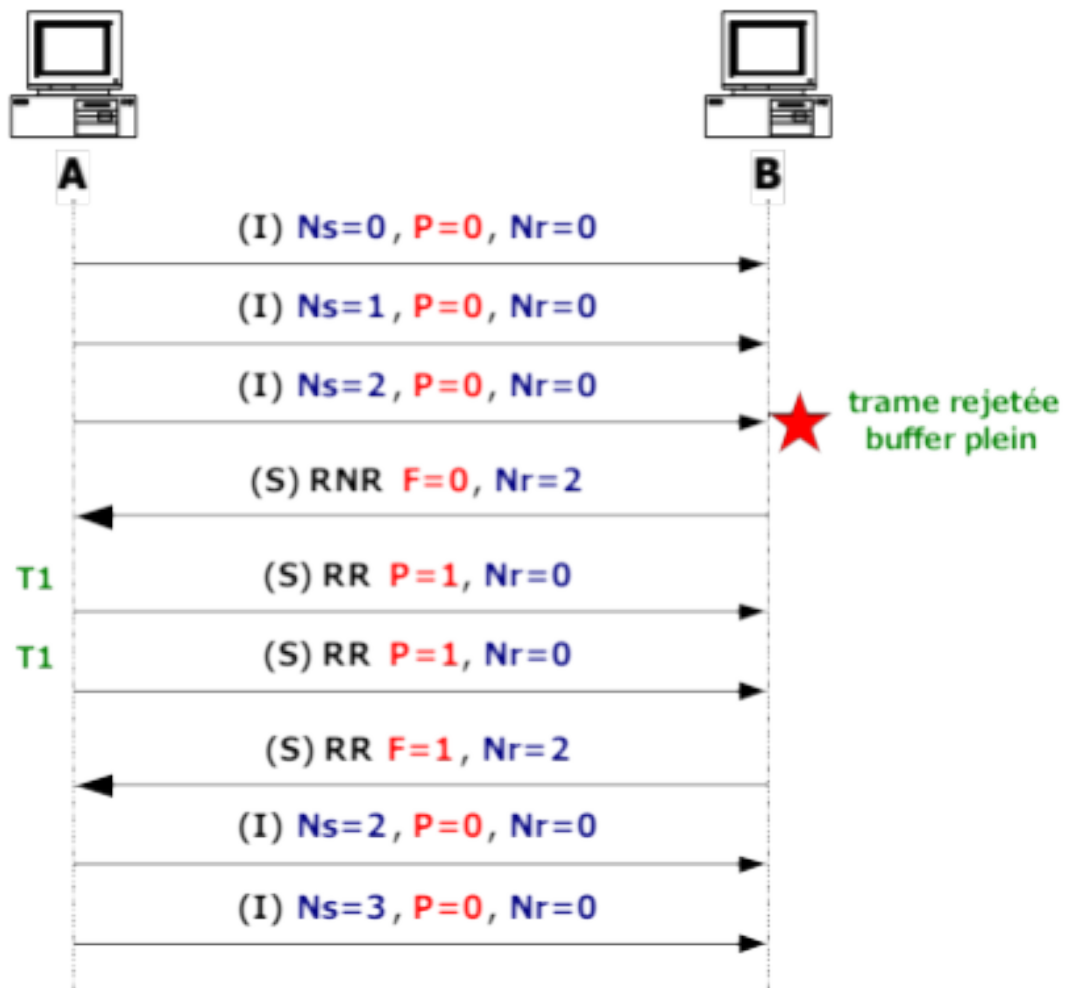
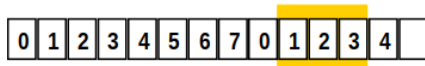
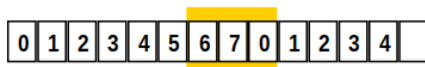
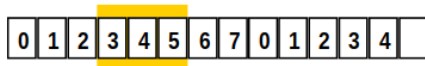
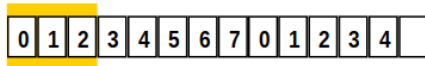


Figure 21: Exemple de gestion du contrôle de flux



■ EXEMPLE $W=3$

- on peut émettre 0, 1, 2
- on reçoit trame RR demandant 3
- on peut émettre 3, 4, 5
- on reçoit trame RR demandant 6
- on peut émettre 6, 7, 0
- on reçoit trame RR demandant 1
- on peut émettre 1, 2, 3
- etc ...

Figure 22: Gestion sautante de la fenêtre

d'acquitter. Cela entraîne une gestion sautante de la fenêtre. Par exemple, si les émissions ne vont que dans un seul sens (de A vers B par exemple), A va stocker dans sa fenêtre d'émission les messages zéro, un, deux, trois, quatre, cinq... Après la réception par B des messages zéro, un et deux, un acquittement est envoyé par B permettant d'acquitter trois messages. L'acquittement suivant libérera de la fenêtre les messages trois, quatre et cinq. Et ainsi de suite... Bien sûr, si des trames I font du "piggybacking" de B vers A, des acquittements intermédiaires seront envoyés.